# Reliable Image Classification with Confidence Rejection

**First working model | Introduction to Computer Vision**

Prepared: 12 October 2026 | Submission deadline: 13 October 2026

**Team:** Aleksandr Medvedev (MedvAx-AI), Telman Nuruzov (Telman3000), Leo Vesin (LeoVesinML), Nikita Shankin (Mysteri0K1ng).

**Current project code:** Public GitHub repository. This report describes main at a882b975.

## 1. Implemented scope

Our objective is a CIFAR-10 classifier that can abstain when confidence is low. This milestone implements reproducible data preparation and a working ten-class baseline: an ImageNet-pretrained ResNet-18 with a trained linear head. The reported experiment uses raw softmax confidence and accepts every prediction. Calibration and rejection will be compared against this fixed baseline later.

## 2. Data loading and preprocessing

The loader downloads the official CIFAR-10 archive, verifies its checksum, and decodes RGB images into 32 x 32 uint8 arrays. CIFAR-10 has 50,000 official training images and 10,000 official test images. We select 17,000 images using seed 42 and save stable source IDs, class counts and hashes in split manifests. The ten classes are airplane, automobile, bird, cat, deer, dog, frog, horse, ship and truck.

| Split | Images | Per class | Purpose / original partition |
|---|---|---|---|
| Training | 10,000 | 1,000 | Head fitting / official training |
| Model validation | 2,000 | 200 | Checkpoint selection and initial results / training |
| Calibration | 1,500 | 150 | Reserved for temperature fitting / training |
| Policy selection | 1,500 | 150 | Reserved for rejection threshold selection / training |
| Final test | 2,000 | 200 | Locked final evaluation / official test |

All five subsets are disjoint. Only the training subset updates model weights. Model validation selects the checkpoint; its results are development estimates. Calibration and policy-selection logits have been exported but have not been used to fit the head. Final-test images have not been evaluated.

The shared sample interface carries the source ID, split, label, corruption, severity and RGB image. The model uses the named pretrained-weight transform: bilinear resize to 256, center crop to 224, conversion to [0,1], then normalization with mean (0.485, 0.456, 0.406) and standard deviation (0.229, 0.224, 0.225). No random augmentation is used.

The implemented corruption generator applies one corruption at native resolution before model preprocessing: Gaussian blur radii 0.5/1.0/1.5, Gaussian noise standard deviations 0.03/0.06/0.12 in [0,1], or JPEG qualities 70/40/15. Noise is deterministically seeded per image and condition. A development sample panel is saved; the quantitative results below cover clean images only.

## 3. Baseline configuration and training

ResNet-18 uses torchvision's **IMAGENET1K_V1** weights. Its original classifier is replaced by a 512-to-10 linear head (5,130 trainable parameters). The backbone is frozen and remains in evaluation mode, including BatchNorm buffers. Clean features are cached for 15,000 development images; only the 10,000 training features train the head.

Training uses cross-entropy, AdamW, learning rate 0.001, weight decay 0.0001, feature batch size 256, seed 42 and a maximum of 30 epochs. Backbone extraction uses batch size 64 on Apple MPS; head optimization runs on CPU. The checkpoint with the lowest model-validation negative log-likelihood (NLL) is selected, breaking ties by the earliest epoch. Epoch 30 was selected. The saved head, configuration, manifests and logits carry hashes for traceability.

## 4. Initial quantitative results

Results come from the committed baseline artifact and predictions, using the selected checkpoint on **2,000 clean model-validation images**, 200 per class. No rejection threshold or temperature calibration is applied (temperature = 1).

| Metric | Result |
|---|---|
| Correct classifications | 1,729 / 2,000 |
| Accuracy | **86.45%** |
| Negative log-likelihood | **0.42448** |
| Expected calibration error (15 bins; fraction) | **0.02465** |
| Coverage / rejection rate | 100% / 0% |

Class accuracies are airplane 88.0%, automobile 90.0%, bird 77.5%, cat 80.0%, deer 84.5%, dog 81.0%, frog 91.0%, horse 88.5%, ship 89.5% and truck 94.5%. Bird is the weakest class and truck the strongest in this development sample. Validation NLL decreased from 0.98321 at epoch 1 to 0.42448 at the selected epoch. Selection at the 30-epoch limit does not establish that training has converged. A single seed and a checkpoint-selection split do not establish final generalization performance.

On the recorded Apple M4 Pro / 48 GB / macOS / Python 3.12.13 environment (PyTorch 2.14.1, torchvision 0.29.1), extracting 15,000 features took **25.45 s**, followed by **0.51 s** for head training on cached features. Full inference took **2.91 s for 1,936 images** (665 images/s), excluding one 64-image warmup batch. This timer includes in-memory image fetching, preprocessing, backbone and head; it excludes dataset download and initial archive loading. These are machine-specific measurements, not promised runtimes on other devices.

## 5. Implementation problems and checks

Windows line-ending conversion initially broke split-file hash verification. LF attributes and consistent newline normalization now make manifest verification portable without changing selected image IDs. Feature caches now require matching file hashes, shapes and extraction provenance; stale or modified caches are rejected. The inference timer was moved before image fetching and remeasured; 2.91 s replaces the earlier incomplete timing.

The reviewed implementation passed 33 automated tests covering data, corruptions, manifests and model behavior. An independent Windows CPU reload of the committed head reproduced all 2,000 predicted labels, with maximum absolute logit difference 0.00002194, below tolerance 0.0001; BatchNorm buffers stayed unchanged. This check reproduces inference, not a fresh training run in the exact pinned macOS environment.

## 6. Current code and reproduction

Data preparation is in `src/data.py`, `src/corruptions.py` and `scripts/prepare_data.py`; the baseline is in `src/model.py`, `src/train.py`, `src/predict.py` and `scripts/train_baseline.py`. Settings are in `configs/data.yaml` and `configs/model.yaml`. The repository includes split manifests, the trained head, clean development logits, predictions and runtime records. Raw data and feature caches are excluded from Git.

From the repository root, install dependencies and verify the committed manifests. The reload command downloads data and pretrained weights when absent, loads the saved head and checks its predictions. To train instead, run the last command without `--verify-reload-only`.

```
python -m pip install -r requirements.txt
python scripts/prepare_data.py --verify-only
python -X utf8 scripts/train_baseline.py --verify-reload-only
```

Evidence: baseline metrics, checkpoint and training history, runtime record.

Telman owns the implemented data pipeline; Leo owns the implemented model and baseline. Nikita owns the next calibration/rejection stage; Aleksandr owns evaluation integration, figures, failure analysis and the demo. Next, we will fit temperature on calibration data, select rejection thresholds on policy-selection data, freeze policies, then evaluate clean and corrupted final-test images. Those later results are not claimed in this submission.